

capa

Introducao

palha sobre o que sao particle systems, para que sao usados, que sao computacionalmente pesados, etc. quisemos usar algoritmos minimamente baseados em fisicas e colicoes. usamos rust e wgpu pois precisavamos de total controlo e queriamos ver como sao as APIs de graficos modernas

Algoritmos de integracao

Para poder simular os movimentos das particulas, precisamos de integrar o mesmo, ou seja, usar passos discretos para simular um movimento que realisticamente seria continuo, baseando-nos nas equacoes de movimento.

O grupo investigou algumas alternativas, tais como:

explicar o que eles sao??

- Basic Verlet Integration
- Velocity Verlet
- Euler integration
- Leapfrog Integration
- Position Verlet
- Runge-Kutta Methods (e.g., RK4)
- Backward Euler

Tinhamos como prioridade obter boa performance mas tambem estabilidade razoavel da simulacao, pelo que optamos por usar Basic Verlet, que nos pareceu um bom compromisso, dadas as comparacoes efetuadas por diversas fontes???????

Collisoes

Algoritmo de colisao

Explicar as contas que sao feitas para colidir duas particulas, dar exemplo grafico

Determinismo

Ao usar um Δt fixo, a simulacao usando Basic Verlet passa a ser deterministica. Esta propriedade pareceu-nos extremamente interessante, visto que o mesmo conjunto de dados de input ira sempre gerar o mesmo output. Assim, por exemplo, poderiamos gerar imagens dando cores as particulas: como elas iriam sempre calhar nos mesmos sitios, as cores iniciais poderiam ser previstas de modo a representarem as cores das imagens.

Otimizacoes

O algoritmo atual percorre N^2 particulas para detetar colisoes, comparando todas as particulas com todas as outras particulas. Isto e muito inefficiente, e podera ser melhorado tirando proveito do facto de que colisoes entre particulas distantes nao tem de ser computadas pois nao irao surtir qualquer efeito. Para alem disso, nao e paralelizavel, deixando de lado possiveis ganhos de performance. Assim, implementamos algumas otimizacoes:

Binning

Para aproveitar o facto de colisões só surtirem efeito se ocorrerem entre partículas próximas, podemos usar um algoritmo de binning, dividindo o espaço numa grelha (em bins) e colidindo apenas partículas entre células adjacentes.

Assim, no início de cada passo de simulação, o algoritmo coloca cada partícula numa célula da grelha com base na sua posição. Para calcular colisões, basta, para cada célula, iterar as suas partículas, calculando colisões apenas com as partículas dentro da própria célula e nas células que a rodeiam.

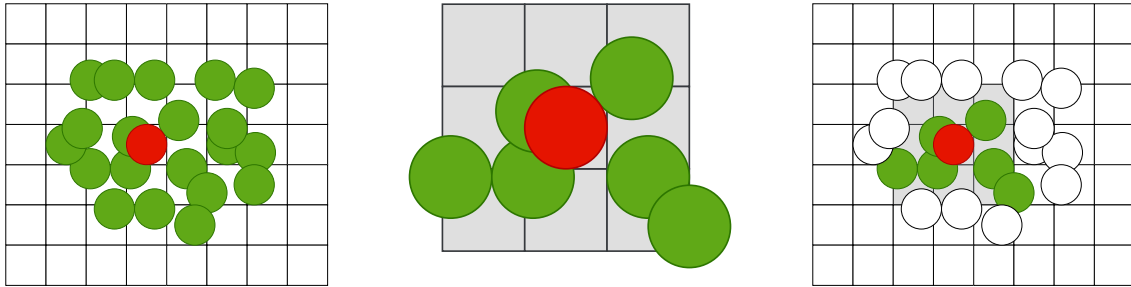


Figure 1: Partículas com que a partícula vermelha precisa de ser comparada.

Esquerda: sem binning. Meio: partículas em células adjacentes. Direita: com binning.

numeros a mostrar o quanto isto melhorou

Multithreading

Visto o espaço agora estar organizado numa grelha, é possível implementar multithreading neste algoritmo, atribuindo uma parte da grelha a cada thread.

No entanto, diferentes threads podem aceder a células adjacentes, provocando data races nas colisões entre as partículas das mesmas. Estas data races tornam o algoritmo não determinístico, o que vai contra o nosso objetivo.

Para voltar a tornar a simulação determinística, decidimos fazer 2 passes de simulação distintos, procurando evitar que threads diferentes possam estar a processar a mesma célula:

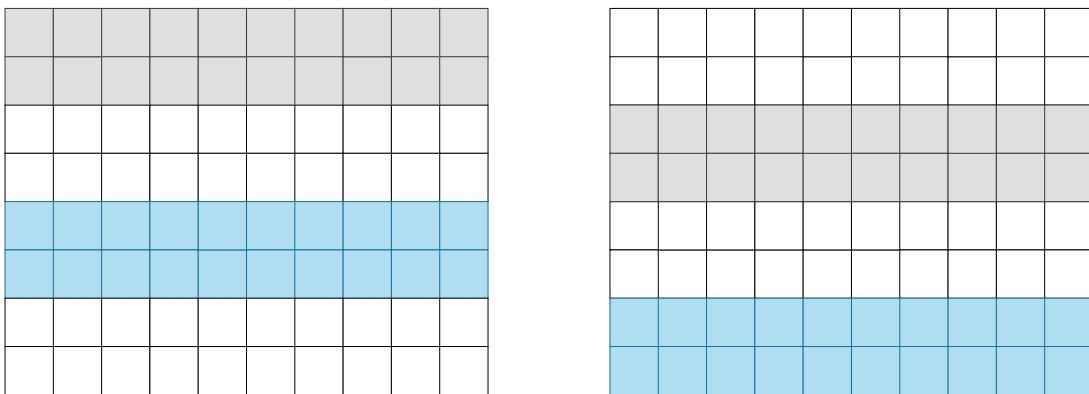


Figure 2: Os espaços cinzento e azul estão a ser processados por duas threads distintas. São feitos dois passes (esquerda e direita), garantindo que ao processar células azuis e cinzentas nunca há células adjacentes a serem processadas em simultâneo

Padding

Ao calcular quais as células que estão em redor de uma dada célula, existem várias posições na grelha que precisam de cuidados especiais, o que complica o código, como as células que se situam nas edges da grelha, e especialmente nos cantos. Seria necessário ter casos especiais em que

determinavamos se se trata de uma destas situacoes, e impedir, por exemplo, a comparacao com celulas acima devido a estas nao existirem.

Assim, decidimos acrescentar uma camada de padding, ou seja, bins que nunca contem qualquer partícula nem sao processadas, de forma a uniformizar o codigo e remover condicoes desnecessarias do mesmo.

Esta otimizacao torna-se especialmente relevante para evitar thread divergence, que sera util para poder executar a detecao de colisoes na GPU no futuro.

Compute shaders

Com o objetivo de aumentar ainda mais o número de partículas na nossa simulação, concluímos que apenas com a capacidade computacional de uma GPU seria possível.

Colisao basica

Numa primeira fase, decidimos implementar novamente o algoritmo ineficiente que compara todas as particulas com todas as outras particulas.

Apesar dos problemas obvios deste algoritmo, ainda assim tivemos enormes ganhos de performance

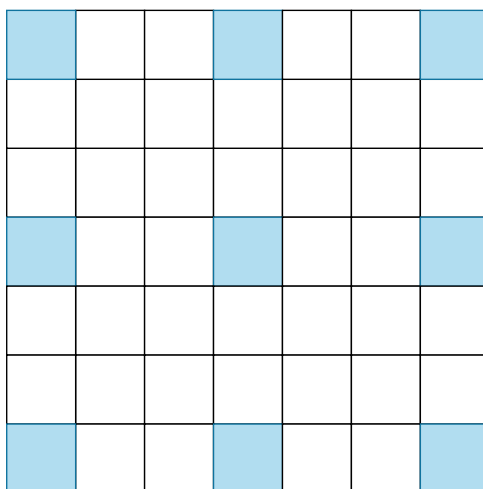
numeros a mostrar o quanto melhorou

Binning

A solucao ideal, usando binning, traz uma nova dimensao aos problemas encontrados na implementacao de multithreading. Tendo uma arquitetura paralela, evitar race conditions torna-se desafiante, mas necessario.

Infelizmente, para manter o determinismo, o passo de gerar as bins tera sempre de ser feito no cpu, ou pelo menos sorted no mesmo, para garantir que as particulas sao sempre processadas na mesma ordem, pelo que nao conseguimos evitar perdas vindas da latencia de transferencia de dados GPU↔CPU.

Nas colisoes, a solucao de usar varios passes diferentes, na solucao de multithreading, tera de ser adaptada a arquitetura da GPU: nao e razoavel que uma thread processe uma zona inteira do espaco da grelha, mas sim uma unica celula. Assim, para cada celula, teremos de garantir que existe um espaco de 2 celulas livre tanto na vertical como na horizontal:



Enquanto na solucao com multithreading seria suficiente 2 passes de simulacao, aqui serao necessarios 9 para que todas as celulas sejam devidamente processadas:

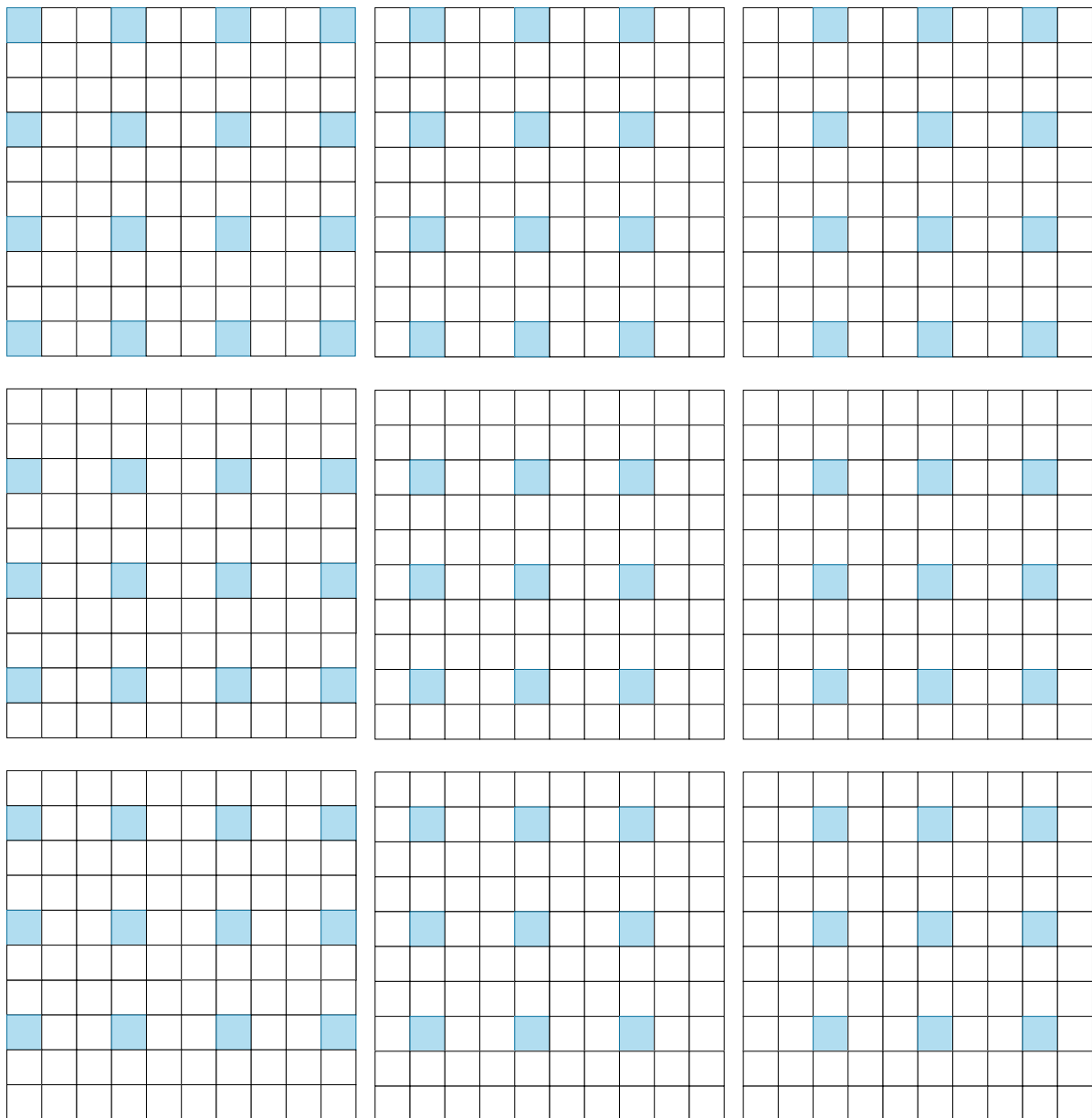


Figure 4: Os 9 passes necessarios para calcular as colisoes de todas as celulas, evitando data races

Renderizacao

Cada particula e composta por 2 triangulos, formando uma quad. Visto todas serem exatamente iguais, apenas instanciamos esta quad para desenhar todas as particulas.

No vertex shader, estas sao redimensionadas, e colocadas na posicao correta do ecrã com base na sua posicao da simulacao. Esta posicao ja esta presente na GPU atraves de um SSBO partilhado com o compute shader, evitando assim a latencia de partilhar dados entre GPU e CPU.

No fragment shader, apenas aplicamos a textura de um circulo para que a particula nao seja renderizada como um quadrado, aplicando tambem uma cor.

mostrar conts de como meter na posicao correta no ecrã?? e explicar como usei pixeis para fazer as coisas

Carregar imagens

Para usarmos a nossa simulacao para criar imagens com as particulas, sera necessario poder atribuir as mesmas uma cor com base na sua posicao.

Assim, quando se pretende carregar uma imagem, fazemos os seguintes passos:

- Decodificar imagem para RGB32
- Redimensionar imagem para a resolucao da grelha de particulas
- Para cada particula, calcular a sua posicao normalizada (0-1) na grelha, e atribuir a cor correspondente da imagem

Com isto, foi possivel atribuir as cores da imagem as particulas:

mostrar uma imagem a ter sido carregada, sem a simulacao ter parado. usar resolucao baixa por agora

Spawners

Para ter maior controlo sobre a criacao de particulas, decidimos criar spawners:

```
struct Spawner {
    start_frame: u64,
    end_frame: u64,

    pos: Vec2,
    dir: Vec2,
    spawner_type: SpawnerType,
}
```

Estes permitem escolher em que frames comecamos e paramos de criar particulas, definir uma posicao e direcao iniciais, bem como usar um SpawnerType para codificar diferentes tipos de spawners. Por exemplo, alguns poderam ser simples e criar particulas num ponto estatico, e outros poderao gera-las num circulo em redor da simulacao.

Com isto, torna-se possivel customizar varias sequencias de criacao de particulas.

Resultados e estabilidade

Com a ajuda do processamento na GPU, foi possivel usar uma grelha 1000*1000, com 1 milhao de particulas:

foto, 1 milhao

No entanto, ao aumentar o numero de particulas para esta escala, as particulas no fundo ficavam sobre bastante “pressao”. A gravidade, ao agir sobre todas as particulas em cima das mesmas, cria colisoes que constantemente empurram as particulas cada vez mais agressivamente para baixo, criando um efeito semelhante a correntes de convexao.

imagem, 1 milhao antes do fix, com desenho das correntes de convexao

Para alem disto, surgia tambem um ponto critico, em que uma particula seria impulsionada com grande velocidade, atingindo outra particula que tambem seria impulsionada, ..., criando um efeito de explosao:

Apesar de um efeito interessante, queriamos obter uma simulacao que chegasse a um estado de descanso, estavel, para que a imagem final fosse perceptivel.

Assim, decidimos introduzir fricção na Verlet Integration. No cálculo da velocidade, introduzimos na mesma um coeficiente de 0 a 1, diminuindo artificialmente a velocidade da partícula, mesmo que esta não esteja em contacto com outras partículas.

De seguida, reduzimos também a própria gravidade, aliviando o problema da pressão. Apesar de não ser realista, a falta de uma escala faz com que não seja perceptível qual o efeito correto ou esperado da gravidade, fazendo com que a simulação continue agradável mesmo com gravidade muito mais fraca.

Por fim, decidimos introduzir substeps para tornar as próprias colisões mais estáveis. Em vez de simular uma vez por frame, com um dado Δt , simulamos N vezes em cada frame, usando

$$\frac{\Delta t}{\text{substeps}}$$

como o novo tempo de simulação, permitindo efetuar as computações em passos mais pequenos, tornando mais improvável que partículas, por se deslocarem demasiado rápido, passem uma por dentro da outra ou penetrem demasiado uma na outra antes que seja detetada uma colisão, gerando uma resposta violenta.

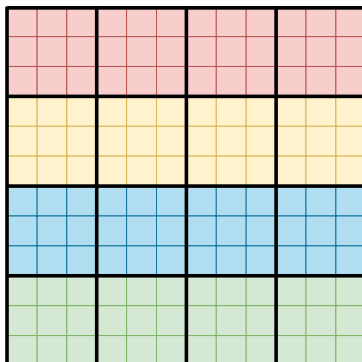
Com estas técnicas, atingimos os nossos objetivos, tendo uma simulação com uma escala satisfatória, determinística e estável, como pode ser visto em [link para video no yt](#)

Trabalho futuro

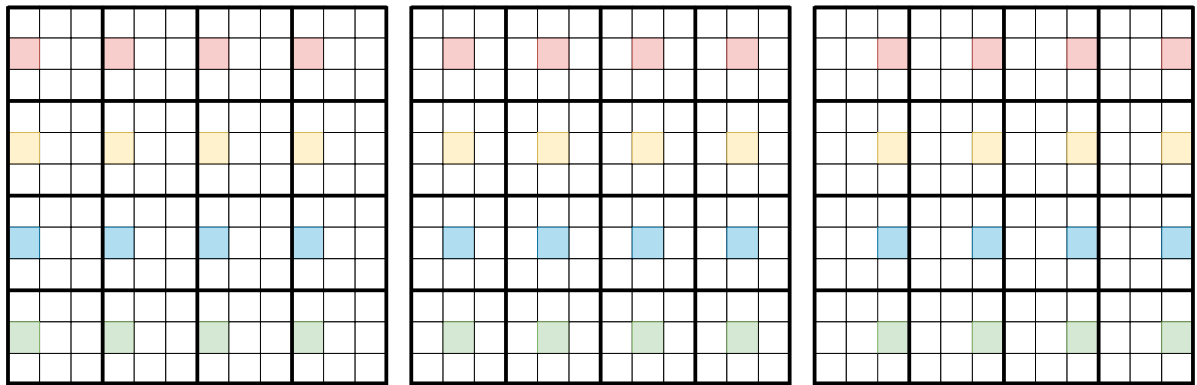
Melhorar computação de colisões na GPU

A técnica mencionada de usar 9 passes de simulação tem algumas ineficiências. Para além da elevada latência na necessidade de sincronizar a GPU 9 vezes, não consideramos a localidade entre as várias threads, abstraindo o conceito de workgroups. Para corrigir isto, concebemos um algoritmo alternativo, que infelizmente não tivemos tempo de implementar, que tira proveito da sincronização ao nível do workgroup:

Cada workgroup fica responsável por uma secção da grelha, e as suas threads processam uma secção 3x3. Neste exemplo, 4 workgroups de cores diferentes possuem 4 threads cada um:



De seguida, fazemos 3 passes, tendo cada um deles 3 pontos de sincronização ao nível do workgroup. Por exemplo, o primeiro passe poderia ser algo como:



Em que entre cada uma das etapas se usa sincronização ao nível do workgroup.

Acreditamos que este algoritmo possa mostrar melhorias de performance, enquanto mantém o determinismo ao evitar race conditions, mas infelizmente não foi implementado.

Melhorar usabilidade dos spawners

Pretendemos, no futuro, usar a modularidade dos spawners para permitir que estes sejam colocados na simulação dinamicamente, através de uma UI.

SoA??

falar do reduce para contar o número de partículas, se eu o implementar